

Experiment Report

Course: Computer Programming

Experiment 2: Selection & Repetition

Class:

Name:

Student ID:

Instructor:

Date: 2026.5.28

Experiment 2: Selection & Repetition

I. Objectives and Content

This experiment aims to practice selection and repetition statements which are the chapters 4-5 of the textbook. The detailed goals of this experiment are as follows.

- Decision Making: Implement one-way, two-way, and multi-way selections using if, else if, and switch statements.
- Iterative Logic: Understand and apply counter-controlled and event-controlled loops.
- Control Flow Mastery: Use selection statements and loop structures to control program execution.
- Data Validation: Utilize do-while loops for robust user input checking.

Complete the following 8 programming tasks. Ensure all code includes proper documentation and follows the specific technical requirements for each problem.

II. Preparation

1. Create a folder named .vscode.
2. Create a .json file under .vscode.
3. Configure task.json to compile, debug, and run the code easily.

4.

III. Project Results

Task 1: Even or Odd Checker (Simple If-Else)

1. **Description:** Detailed descriptions of the code.

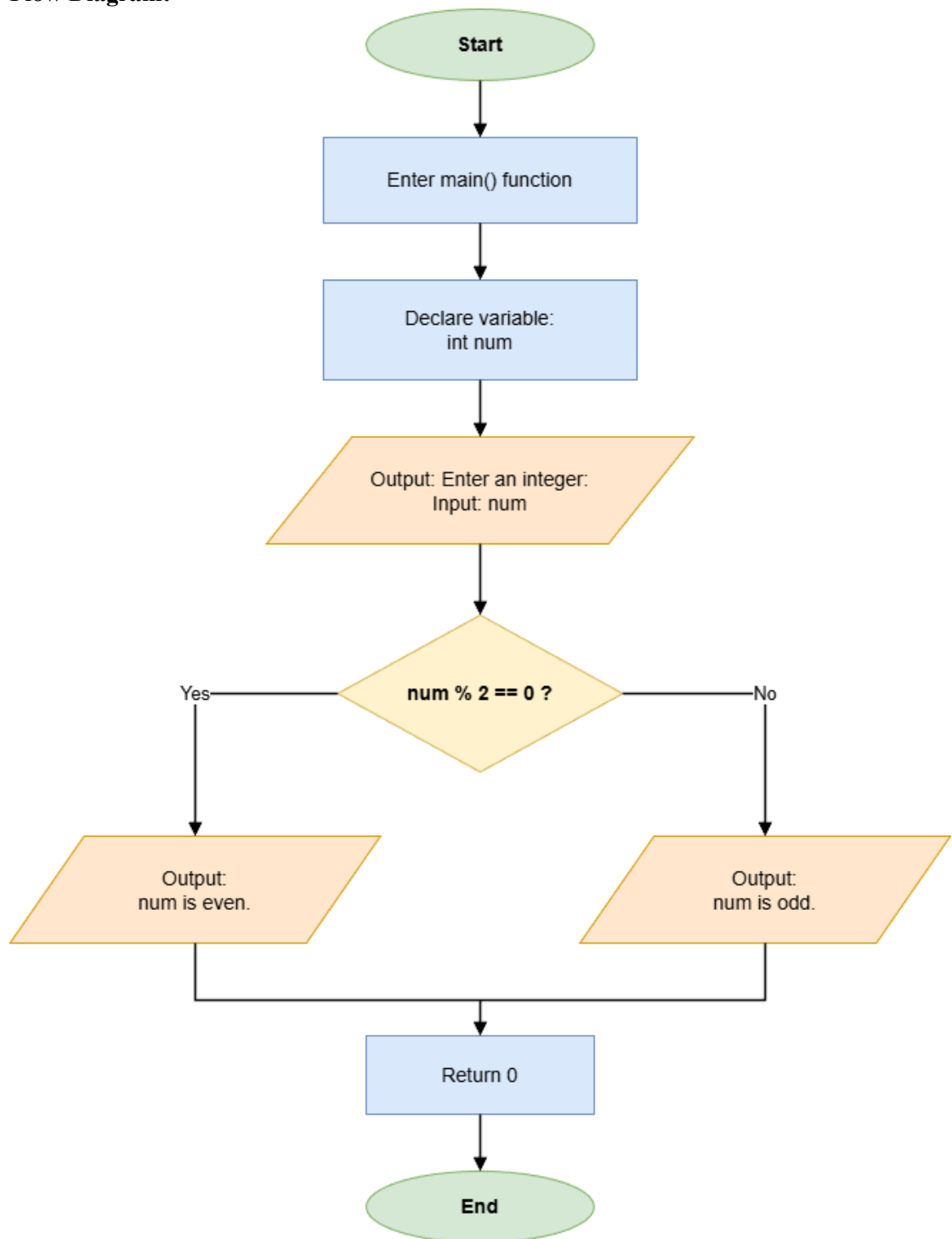
1) Input the integer.

```
int num;  
  
cout << "Enter a integer: ";  
cin >> num;
```

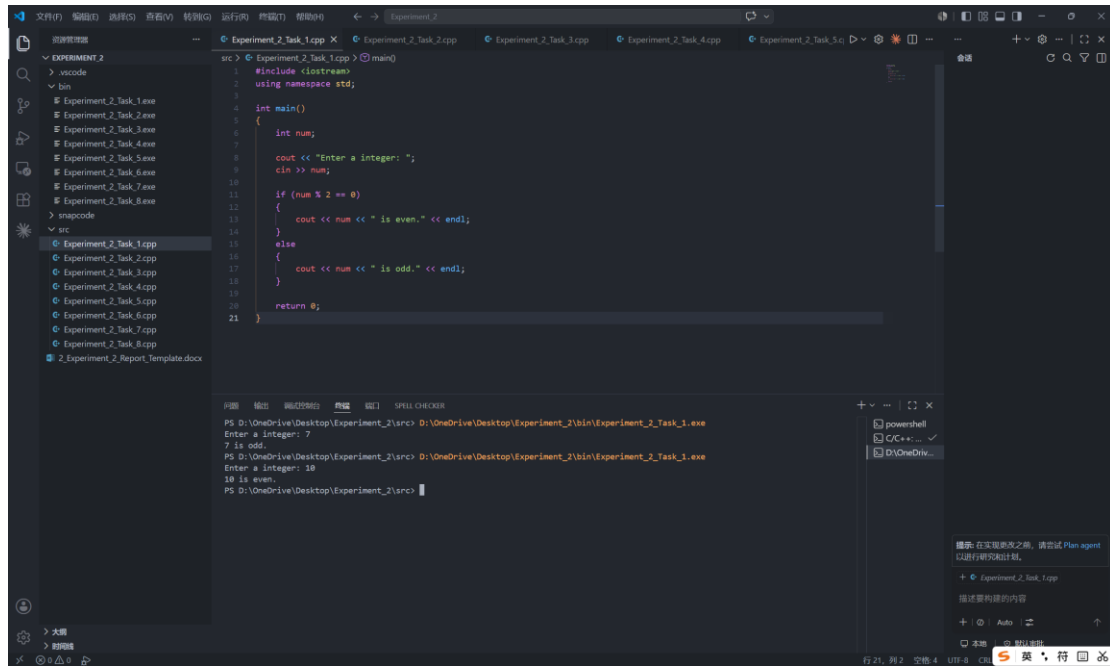
2) Check the integer is odd or even.

```
if (num % 2 == 0)  
{  
    cout << num << " is even." << endl;  
}  
else  
{  
    cout << num << " is odd." << endl;  
}
```

2. Flow Diagram:



3. Results (Output Screenshot): The screenshot of the output of the code.



The screenshot displays the Visual Studio Code interface. The left sidebar shows a file explorer with a project named 'EXPERIMENT_2' containing several source files and executables. The main editor window shows a C++ program in 'Experiment_2_Task_1.cpp'. The code includes `<iostream>` and `using namespace std;`, and defines a `main()` function that prompts the user to enter an integer, checks if it is even or odd, and prints the result. The bottom panel shows the output of the program, which matches the code's logic: for input 7, it outputs '7 is odd.', and for input 10, it outputs '10 is even.'. The status bar at the bottom indicates the file is at line 21, column 4, with UTF-8 encoding.

```
1 #include <iostream>
2 using namespace std;
3
4 int main()
5 {
6     int num;
7
8     cout << "Enter a integer: ";
9     cin >> num;
10
11     if (num % 2 == 0)
12     {
13         cout << num << " is even." << endl;
14     }
15     else
16     {
17         cout << num << " is odd." << endl;
18     }
19
20     return 0;
21 }
```

输出

```
PS D:\OneDrive\Desktop\Experiment_2\src> D:\OneDrive\Desktop\Experiment_2\bin\Experiment_2_Task_1.exe
Enter a integer: 7
7 is odd.
PS D:\OneDrive\Desktop\Experiment_2\src> D:\OneDrive\Desktop\Experiment_2\bin\Experiment_2_Task_1.exe
Enter a integer: 10
10 is even.
PS D:\OneDrive\Desktop\Experiment_2\src>
```

Task 2: Absolute Value & Range Logic

1. Description:

- 1) Input a number.

```
int num;  
  
cout << "Enter a number: ";  
cin >> num;
```

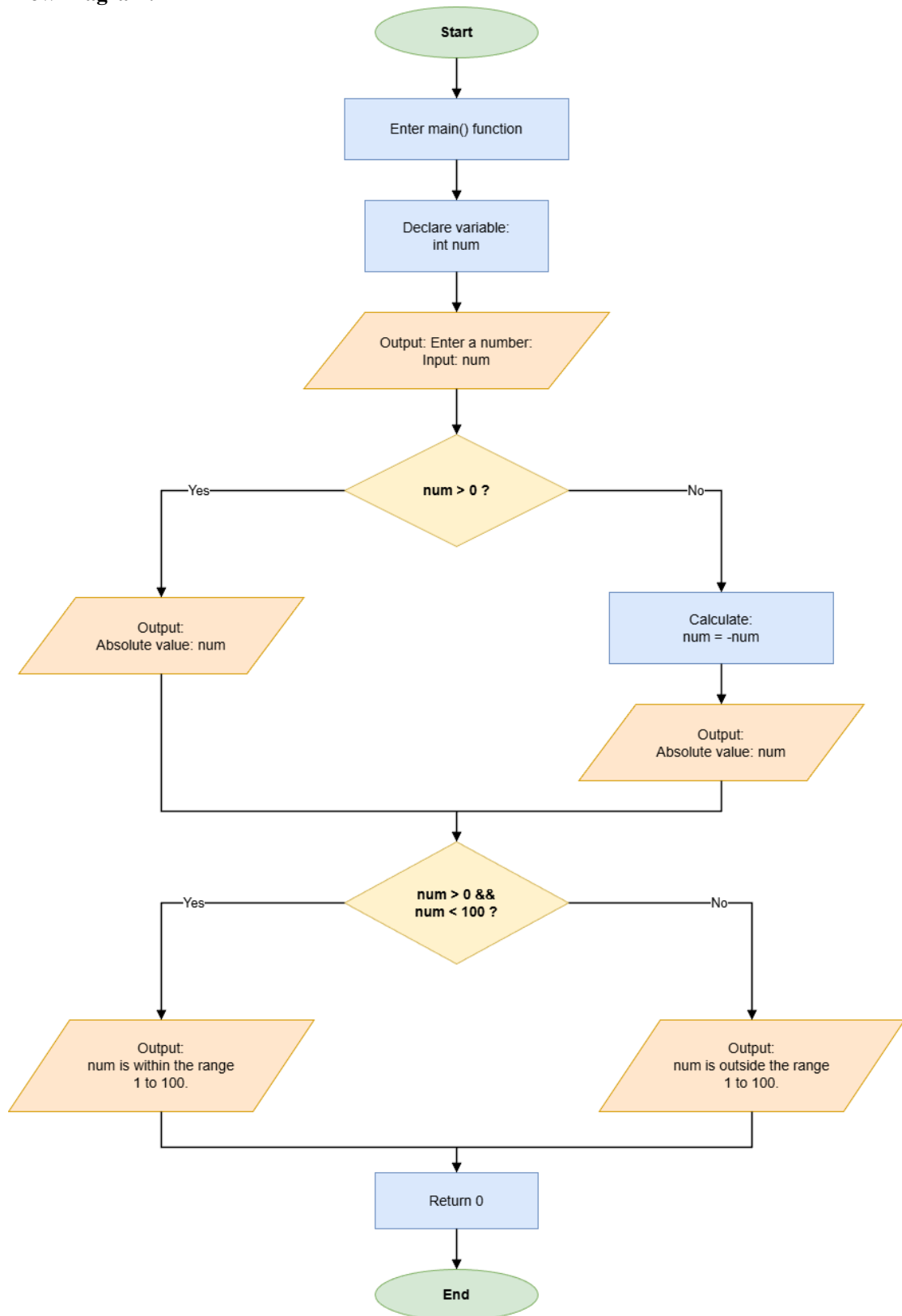
- 2) Print the absolute value.

```
if (num > 0)  
{  
    cout << "Absolute value: " << num << endl;  
}  
else  
{  
    num = -num;  
    cout << "Absolute value: " << num << endl;  
}
```

- 3) Check if the value is between 1 and 100.

```
if (num > 0 && num < 100)  
{  
    cout << num << " is within the range 1 to 100." << endl;  
}  
else  
{  
    cout << num << " is outside the range 1 to 100." << endl;  
}
```

2. Flow Diagram:



3. Results (Output Screenshot):

```
src > Experiment_2_Task_2.cpp > main()
10  cin >> num;
11
12  if (num > 0)
13  {
14      cout << "Absolute value: " << num << endl;
15  }
16  else
17  {
18      num = -num;
19      cout << "Absolute value: " << num << endl;
20  }
21
22  if (num > 0 && num < 100)
23  {
24      cout << num << " is within the range 1 to 100." << endl;
25  }
26  else
27  {
28      cout << num << " is outside the range 1 to 100." << endl;
29  }
30  return 0;
31 }
```

```
PS D:\OneDrive\Desktop\Experiment_2\src> D:\OneDrive\Desktop\Experiment_2\bin\Experiment_2_Task_2.exe
Enter a number: -45
Absolute value: 45
45 is within the range 1 to 100.
PS D:\OneDrive\Desktop\Experiment_2\src> D:\OneDrive\Desktop\Experiment_2\bin\Experiment_2_Task_2.exe
Enter a number: 131
Absolute value: 131
131 is outside the range 1 to 100.
PS D:\OneDrive\Desktop\Experiment_2\src>
```

Task 3: Simple Calculator (Switch Case)

1. Description:

- 1) Input the two numbers and the operator.

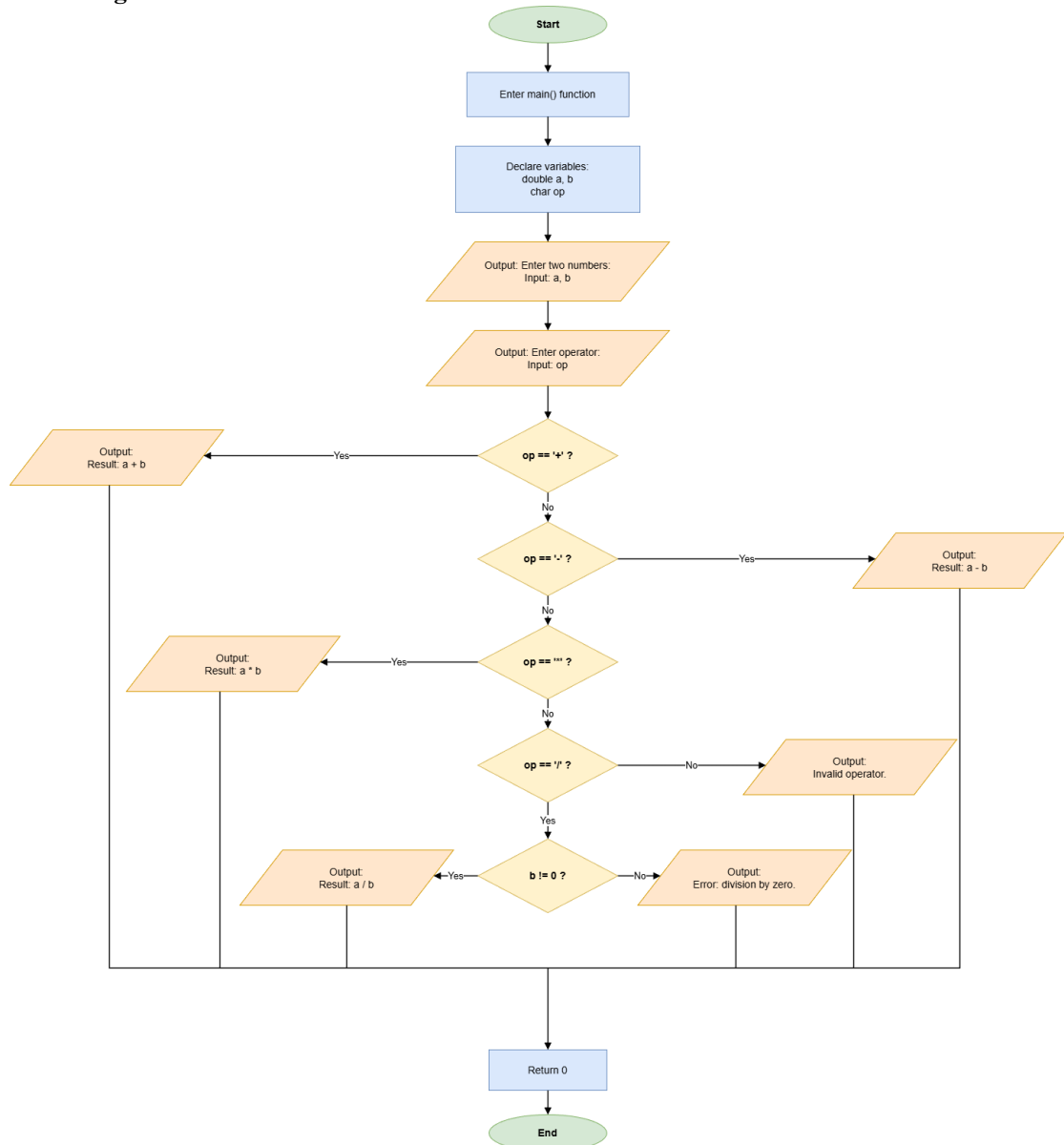
```
double a, b;
char op;

cout << "Enter two numbers: ";
cin >> a >> b;
cout << "Enter operator :";
cin >> op;
```

- 2) Calculate using switch-case.

```
switch (op)
{
case '+':
    cout << "Result: " << a + b << endl;
    break;
case '-':
    cout << "Result: " << a - b << endl;
    break;
case '*':
    cout << "Result: " << a * b << endl;
    break;
case '/':
    if (b != 0)
    {
        cout << "Result: " << a / b << endl;
    }
    else
    {
        cout << "Error: division by zero." << endl;
    }
    break;
default:
    cout << "Invalid operator." << endl;
}
```

2. Flow Diagram:



3. Results (Output Screenshot):

The screenshot displays the Visual Studio Code interface with a C++ project named 'Experiment_2'. The file explorer on the left shows a directory structure with 'src' and 'bin' folders, and several task files. The main editor window shows the source code for 'Experiment_2_Task_3.cpp', which includes `<iostream>` and `using namespace std;`. The `main` function prompts the user to enter two numbers and an operator, then uses a `switch` statement to perform addition, subtraction, multiplication, or division. The output console at the bottom shows the program's execution: it prompts for two numbers (10 and 5) and an operator (*), resulting in 'Result: 50'. A second run prompts for 10 and 0 with the '/' operator, resulting in an 'Error: division by zero.' message.

```
src > Experiment_2_Task_3.cpp > main()
1 #include <iostream>
2 using namespace std;
3
4 int main()
5 {
6     double a, b;
7     char op;
8
9     cout << "Enter two numbers: ";
10    cin >> a >> b;
11    cout << "Enter operator :";
12    cin >> op;
13
14    switch (op)
15    {
16    case '+':
17        cout << "Result: " << a + b << endl;
18        break;
19    case '-':
20        cout << "Result: " << a - b << endl;
21        break;
22    case '*':
23        cout << "Result: " << a * b << endl;
24        break;
25    case '/':
26        if (b != 0)
27        {
28            cout << "Result: " << a / b << endl;
29        }
30    }
```

输出

```
PS D:\OneDrive\Desktop\Experiment_2\src> D:\OneDrive\Desktop\Experiment_2\bin\Experiment_2_Task_3.exe
Enter two numbers: 10 5
Enter operator : *
Result: 50
PS D:\OneDrive\Desktop\Experiment_2\src> D:\OneDrive\Desktop\Experiment_2\bin\Experiment_2_Task_3.exe
Enter two numbers: 10 0
Enter operator : /
Error: division by zero.
PS D:\OneDrive\Desktop\Experiment_2\src>
```

Task 4: Score to Grade Converter (Multi-way Selection)

1. Description:

- 1) Input the score.

```
int score;

cout << "Enter score: ";
cin >> score;
```

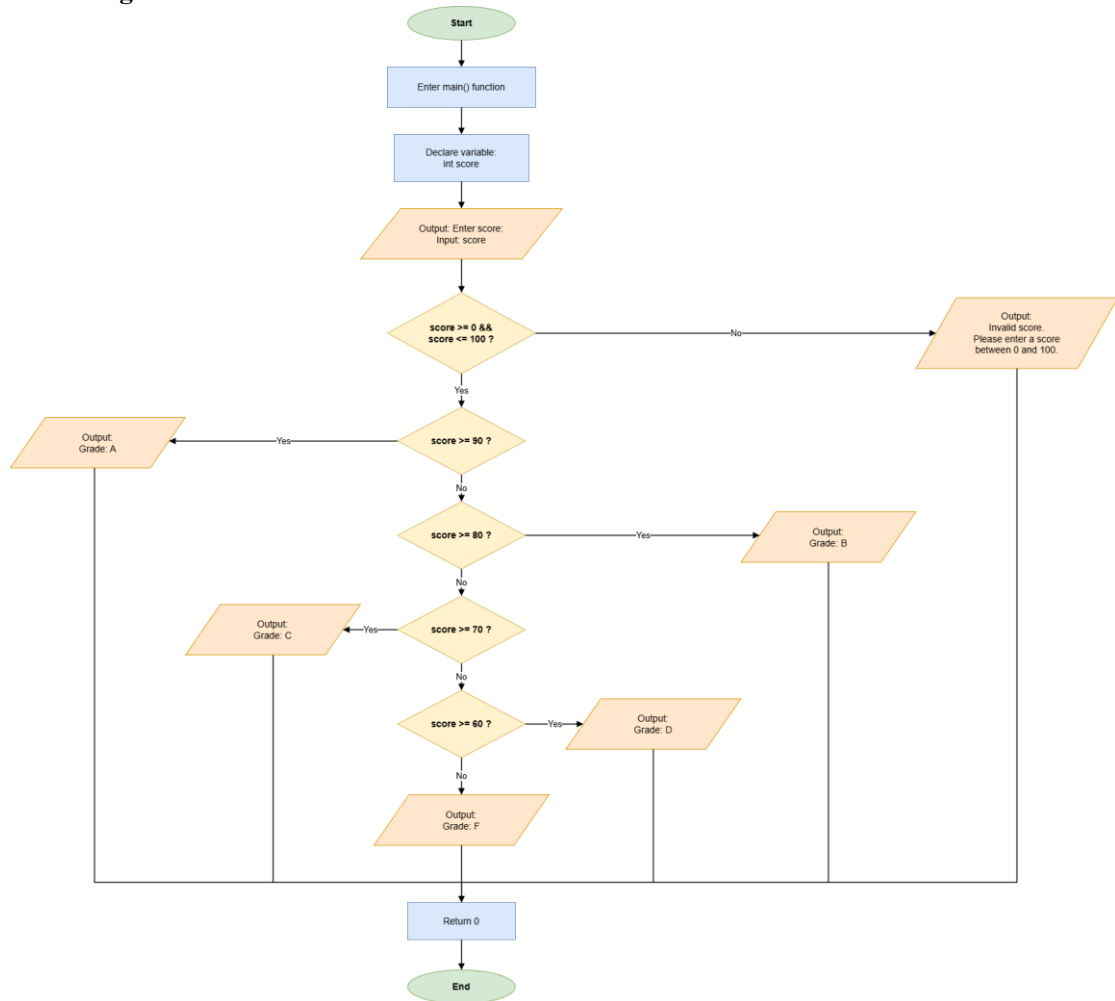
- 2) Output the letter grade using if-else chain.

```
if (score >= 0 && score <= 100)
{
    if (score >= 90)
    {
        cout << "Grade: A" << endl;
    }
    else if (score >= 80)
    {
        cout << "Grade: B" << endl;
    }
    else if (score >= 70)
    {
        cout << "Grade: C" << endl;
    }
    else if (score >= 60)
    {
        cout << "Grade: D" << endl;
    }
    else
    {
        cout << "Grade: F" << endl;
    }
}
```

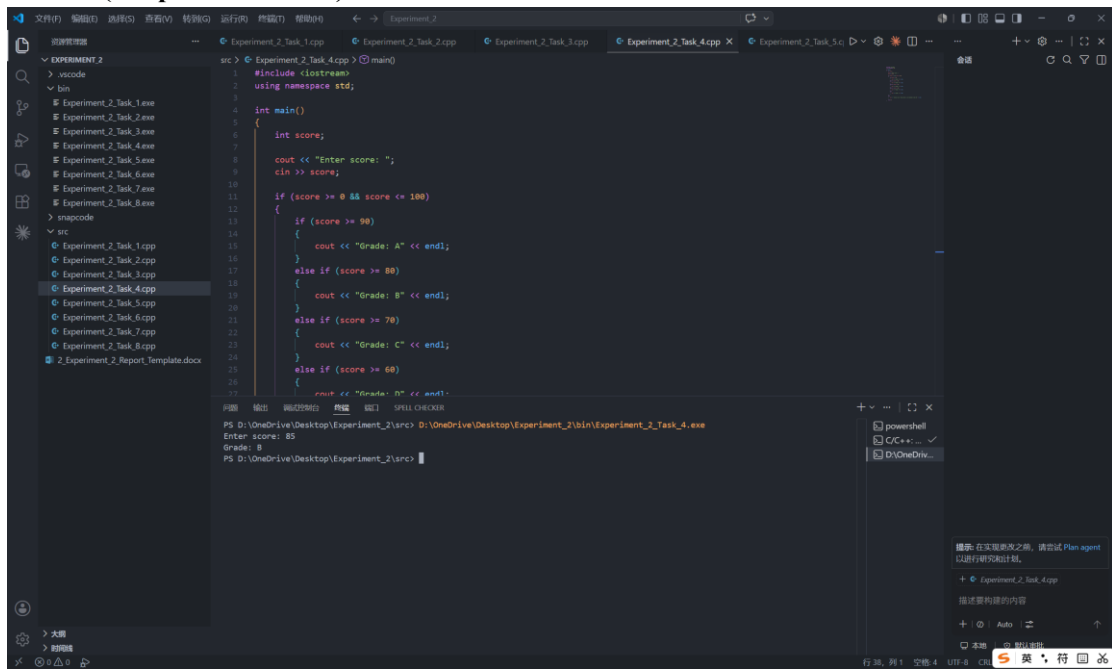
- 3) Validate the input is within the 0-100 range.

```
else
{
    cout << "Invalid score. Please enter a score between 0 and 100." << endl;
}
```

2. Flow Diagram:



3. Results (Output Screenshot):



The screenshot displays the Visual Studio Code editor with a C++ source file named `Experiment_2_Task_4.cpp`. The code implements a program that prompts the user to enter a score and then outputs a corresponding grade based on a set of conditions.

```
1 #include <iostream>
2 using namespace std;
3
4 int main()
5 {
6     int score;
7
8     cout << "Enter score: ";
9     cin >> score;
10
11     if (score >= 0 && score <= 100)
12     {
13         if (score >= 90)
14         {
15             cout << "Grade: A" << endl;
16         }
17         else if (score >= 80)
18         {
19             cout << "Grade: B" << endl;
20         }
21         else if (score >= 70)
22         {
23             cout << "Grade: C" << endl;
24         }
25         else if (score >= 60)
26         {
27             cout << "Grade: D" << endl;
28         }
29     }
30 }
```

The output window at the bottom shows the execution results:

```
PS D:\OneDrive\Desktop\Experiment_2\src> D:\OneDrive\Desktop\Experiment_2\bin\Experiment_2_Task_4.exe
Enter score: 85
Grade: B
PS D:\OneDrive\Desktop\Experiment_2\src>
```

The interface also includes a file explorer on the left showing the project structure, a search bar, and a status bar at the bottom indicating the current file is `Experiment_2_Task_4.cpp` at line 30, column 4.

Task 5: Sum of N Integers (For Loop)

1. Description:

- 1) Input a positive integer N.

```
int N, i, sum = 0;
cout << "Enter N: ";
cin >> N;
```

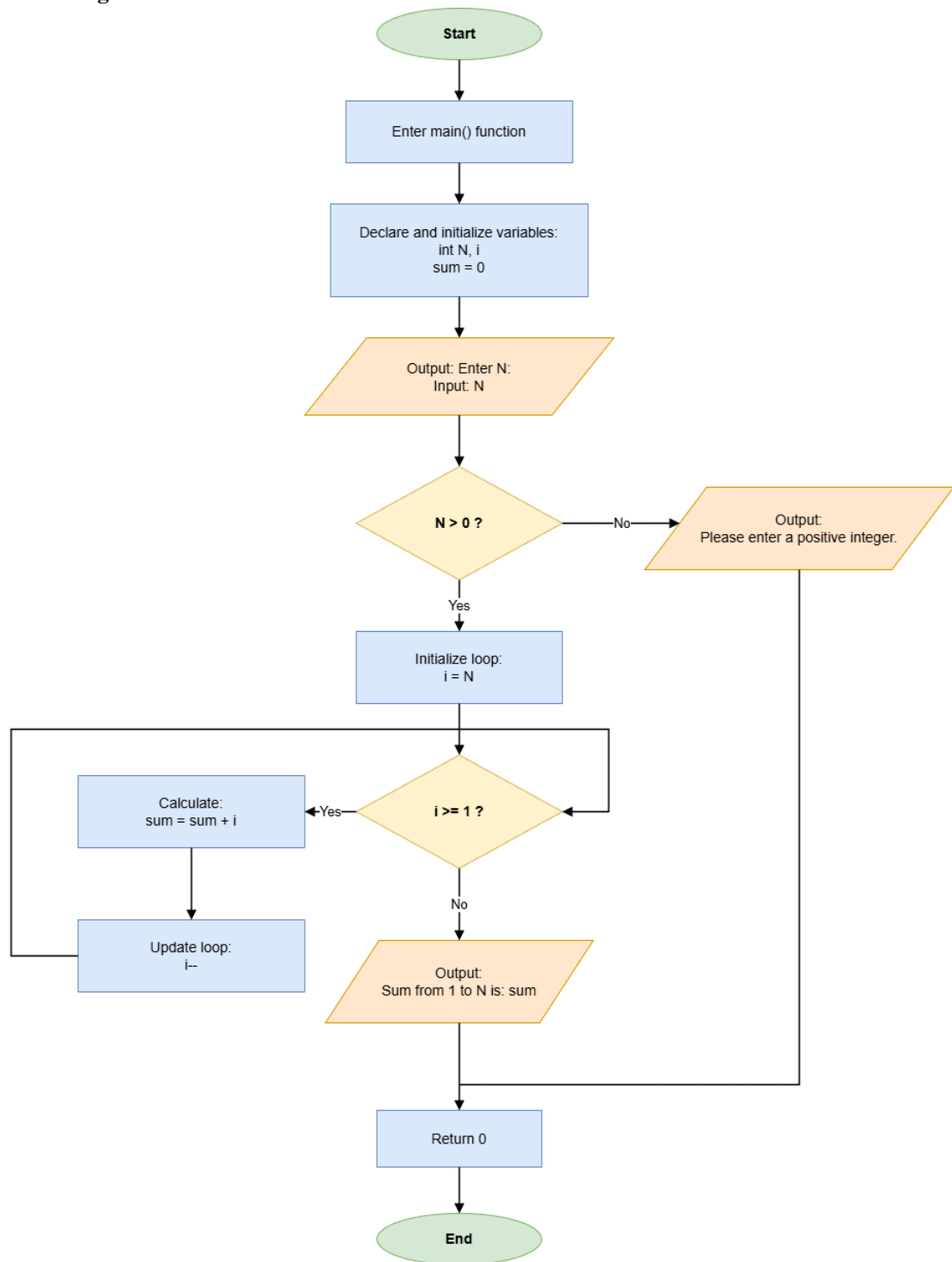
- 2) Calculate the sum of all integers from 1 to N.

```
if (N > 0)
{
    for (i = N; i >= 1; i--)
    {
        sum = sum + i;
    }
    cout << "Sum from 1 to " << N << " is: " << sum << endl;
}
```

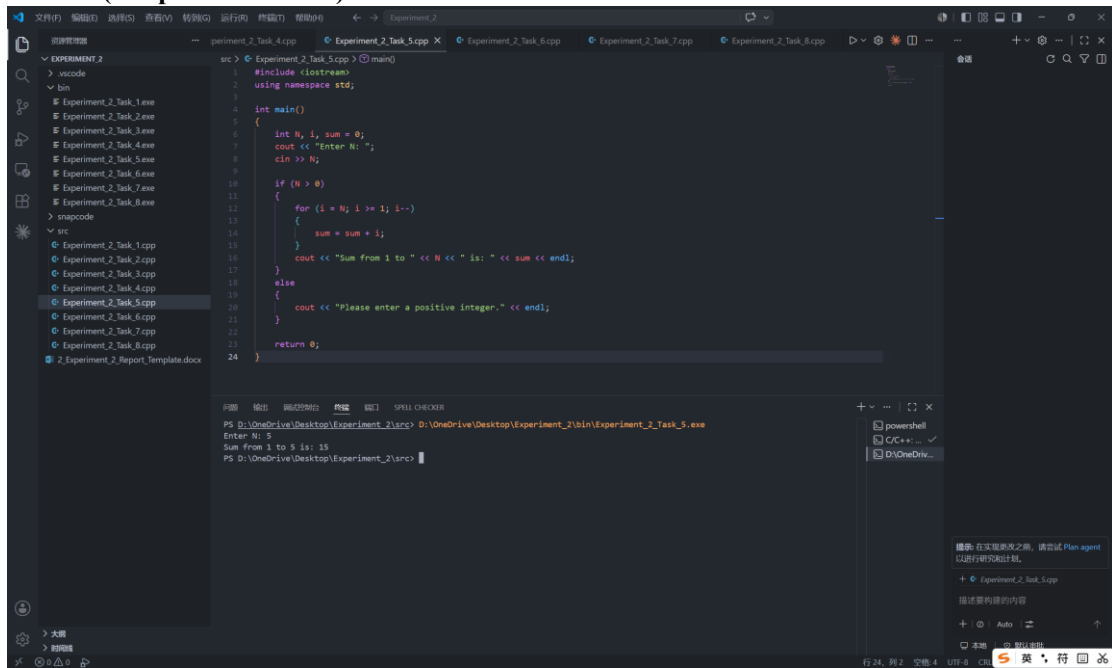
- 3) Check to ensure that the input is not a negative number.

```
else
{
    cout << "Please enter a positive integer." << endl;
}
```

2. Flow Diagram:



3. Results (Output Screenshot):



The screenshot displays the Visual Studio Code editor with the file `Experiment_2_Task_5.cpp` open. The code is as follows:

```
1 #include <iostream>
2 using namespace std;
3
4 int main()
5 {
6     int N, i, sum = 0;
7     cout << "Enter N: ";
8     cin >> N;
9
10    if (N > 0)
11    {
12        for (i = N; i >= 1; i--)
13        {
14            sum = sum + i;
15        }
16        cout << "Sum from 1 to " << N << " is: " << sum << endl;
17    }
18    else
19    {
20        cout << "Please enter a positive integer." << endl;
21    }
22    return 0;
23 }
```

The output window at the bottom shows the execution results:

```
PS D:\OneDrive\Desktop\Experiment_2\src> D:\OneDrive\Desktop\Experiment_2\bin\Experiment_2_Task_5.exe
Enter N: 5
Sum from 1 to 5 is: 15
PS D:\OneDrive\Desktop\Experiment_2\src>
```

The interface also shows a file explorer on the left with the project structure, a search bar, and a status bar at the bottom indicating the current file is `Experiment_2_Task_5.cpp` and the encoding is `UTF-8`.

Task 6: Input Validation (Do-While Loop)

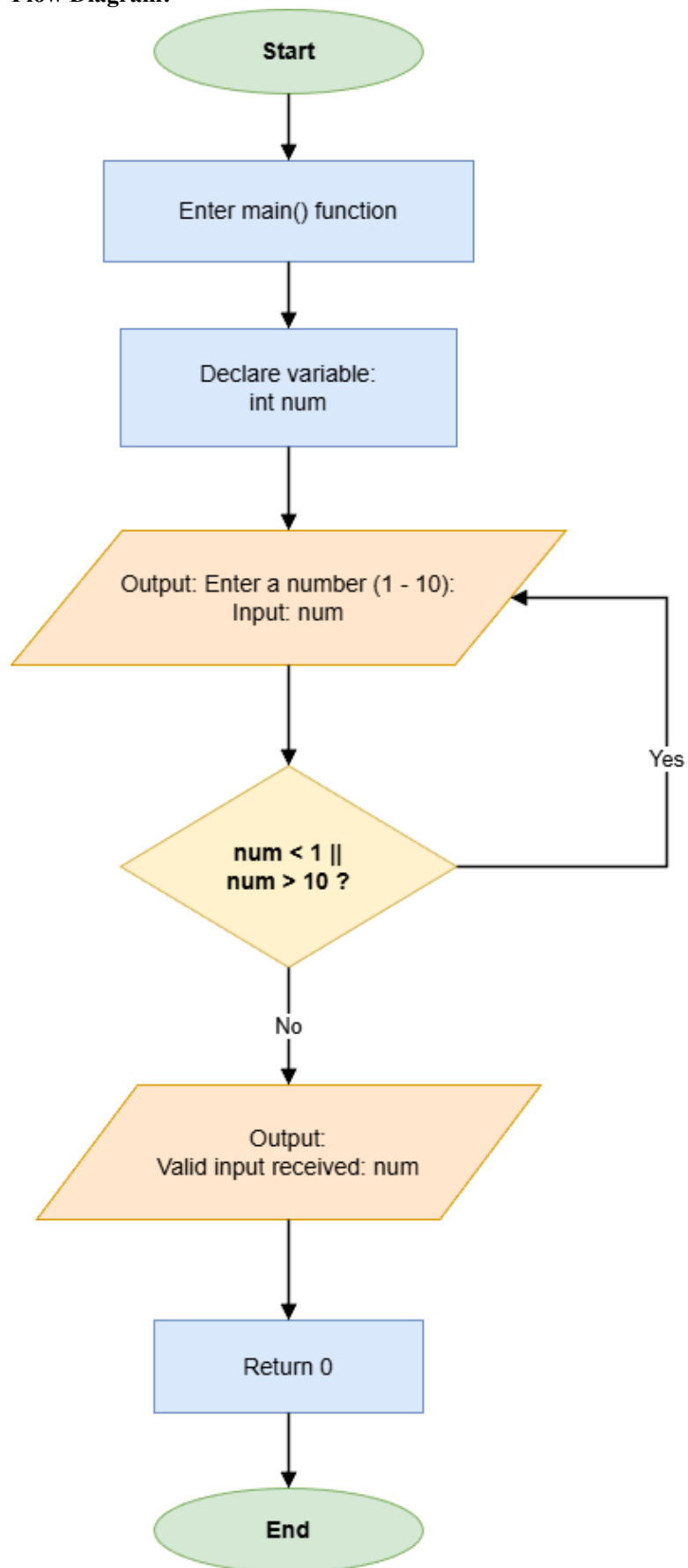
1. Description:

- 1) Repeatedly ask the user to enter a number between 1 and 10 until a valid number is entered.

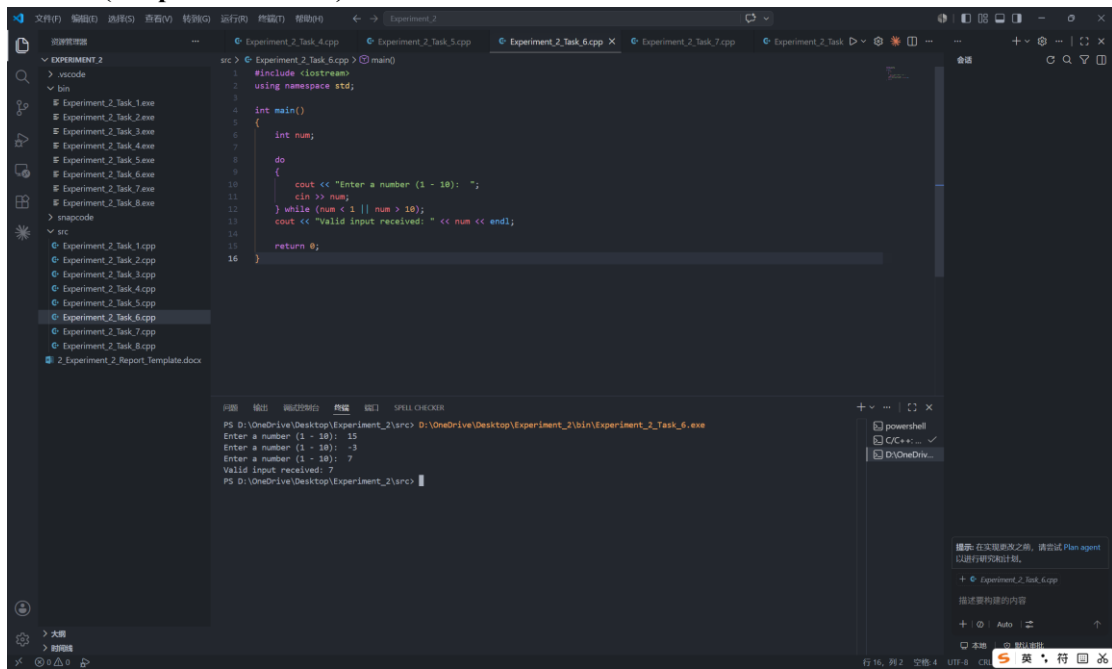
```
int num;

do
{
    cout << "Enter a number (1 - 10): ";
    cin >> num;
} while (num < 1 || num > 10);
cout << "Valid input received: " << num << endl;
```

2. Flow Diagram:



3. Results (Output Screenshot):



The screenshot displays the Visual Studio Code interface. The left sidebar shows a file explorer with a project named 'EXPERIMENT_2' containing various task files. The main editor window shows the source code for 'Experiment_2_Task_6.cpp'. The code is a C++ program that prompts the user to enter a number between 1 and 10. It uses a do-while loop to validate the input. The output window at the bottom shows the program's execution, displaying the prompts and the user's input: 15, -3, 7, and the final output 'Valid input received: 7'.

```
src > Experiment_2_Task_6.cpp > main()
1 #include <iostream>
2 using namespace std;
3
4 int main()
5 {
6     int num;
7
8     do
9     {
10         cout << "Enter a number (1 - 10): ";
11         cin >> num;
12     } while (num < 1 || num > 10);
13     cout << "Valid input received: " << num << endl;
14
15     return 0;
16 }
```

终端 输出 调试控制台 性能 错误 SPELL CHECKER

```
PS D:\OneDrive\Desktop\Experiment_2\src> D:\OneDrive\Desktop\Experiment_2\bin\Experiment_2_Task_6.exe
Enter a number (1 - 10): 15
Enter a number (1 - 10): -3
Enter a number (1 - 10): 7
Valid input received: 7
PS D:\OneDrive\Desktop\Experiment_2\src>
```

Task 7: Sentinel-Controlled Average

1. Description:

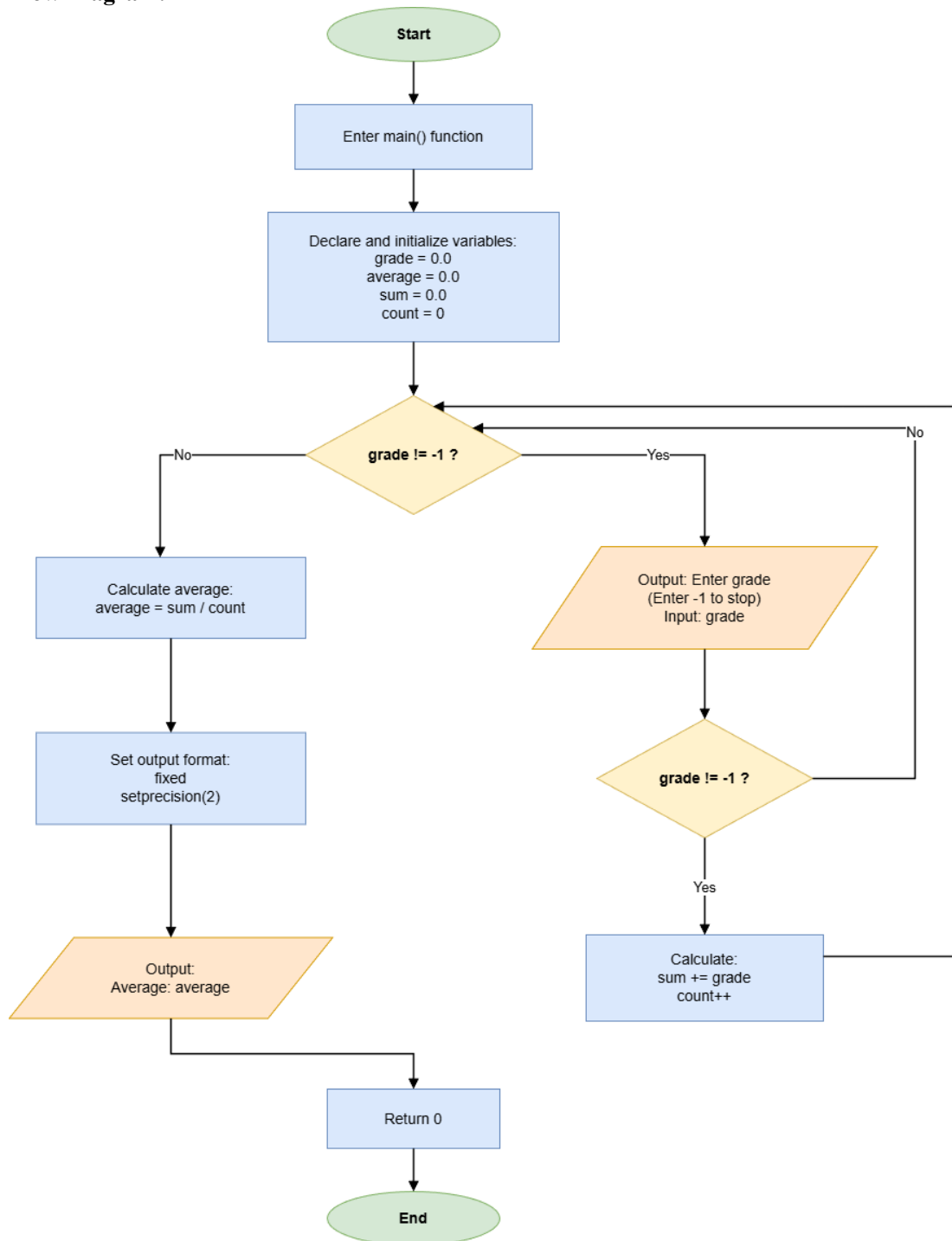
- 1) Enter multiple grades until -1 is entered.

```
while (grade != -1)
{
    cout << "Enter grade (Enter -1 to stop): ";
    cin >> grade;
    if (grade != -1)
    {
        sum += grade;
        count++;
    }
}
```

- 2) Calculate the average excluding -1.

```
average = sum / count;
cout << fixed << setprecision(2);
cout << "Average: " << average << endl;
```

2. Flow Diagram:



3. Results (Output Screenshot):

The screenshot displays a C++ IDE with the following components:

- File Explorer (Left):** Shows a project structure with folders 'src' and 'bin', and various task-related files.
- Source Code (Center):** The file `Experiment_2_Task_7.cpp` is open, showing the following code:

```
1 #include <iostream>
2 #include <iomanip>
3 using namespace std;
4
5 int main()
6 {
7     double grade = 0.0, average = 0.0, sum = 0.0;
8     int count = 0;
9     while (grade != -1)
10     {
11         cout << "Enter grade (Enter -1 to stop): ";
12         cin >> grade;
13         if (grade != -1)
14         {
15             sum += grade;
16             count++;
17         }
18     }
19     average = sum / count;
20     cout << fixed << setprecision(2);
21     cout << "Average: " << average << endl;
22
23     return 0;
24 }
```
- Console Output (Bottom):** The output of the program is shown:

```
PS D:\OneDrive\Desktop\Experiment_2\src> D:\OneDrive\Desktop\Experiment_2\bin\Experiment_2_Task_7.exe
Enter grade (Enter -1 to stop): 58
Enter grade (Enter -1 to stop): 68.5
Enter grade (Enter -1 to stop): 91
Enter grade (Enter -1 to stop): -1
Average: 72.50
PS D:\OneDrive\Desktop\Experiment_2\src>
```
- Right Panel:** Contains a search bar and a list of files, including `Experiment_2_Task_7.cpp`.

Task 8: Multiplication Table (Nested Loops)

1. Description:

- 1) Print the title.

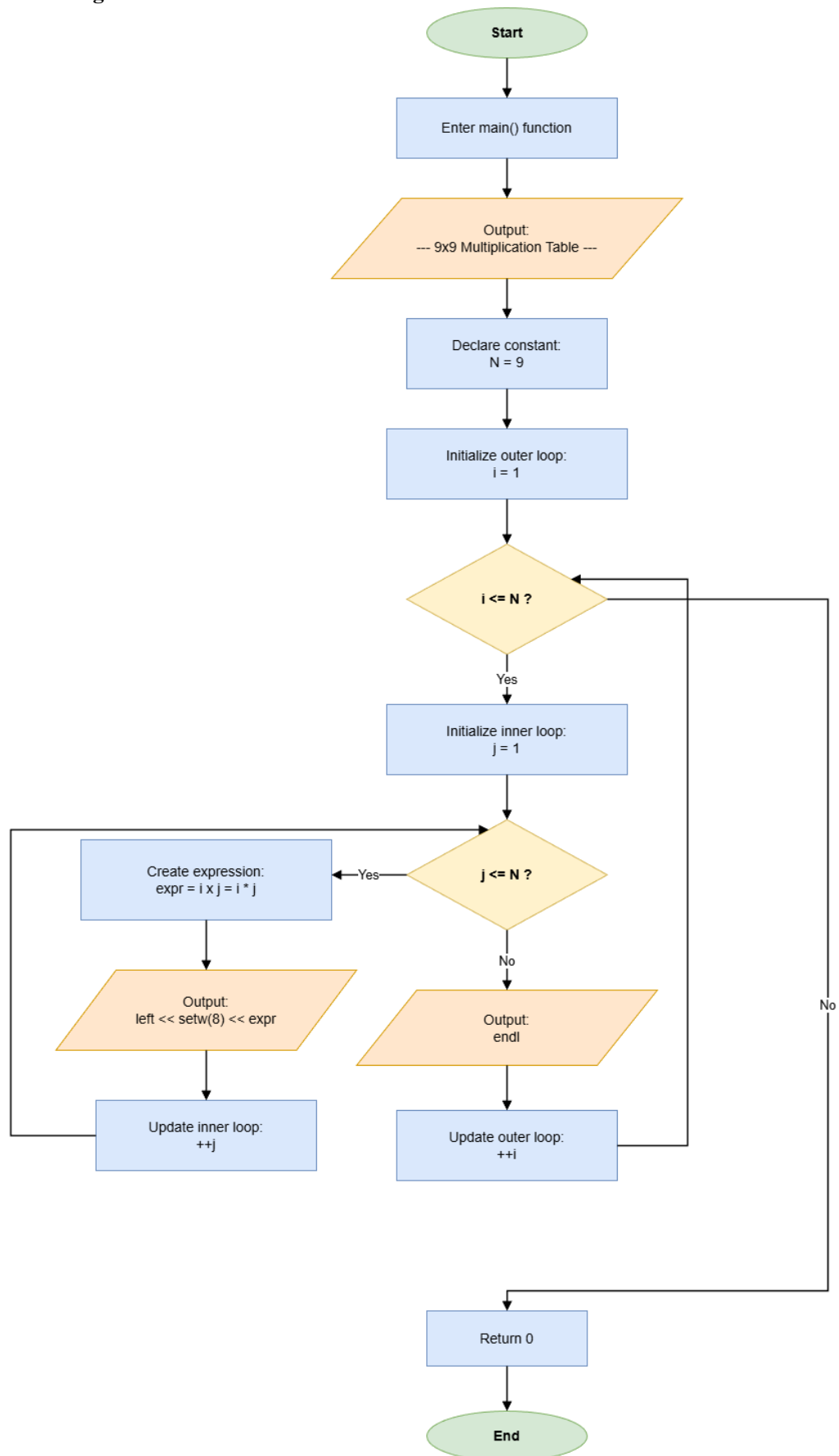
```
cout << "--- 9x9 Multiplication Table ---" << endl;
```

- 2) Use nested loops and setw for alignment.

```
const int N = 9;

for (int i = 1; i <= N; ++i)
{
    for (int j = 1; j <= N; ++j)
    {
        string expr = to_string(i) + "x" + to_string(j) + "=" +
to_string(i * j);
        cout << left << setw(8) << expr;
    }
    cout << endl;
}
```

2. Flow Diagram:



3. Results (Output Screenshot):

The screenshot shows a C++ IDE with a project named 'Experiment_2'. The source file 'Experiment_2_Task_8.cpp' is open, displaying a program that prints a 9x9 multiplication table. The code uses nested loops and the `setw` function for formatting. The output window shows the execution result, which is a 9x9 grid of multiplication results, formatted with columns of width 8.

```
src > Experiment_2_Task_8.cpp > main()
1 #include <iostream>
2 #include <iomanip>
3 using namespace std;
4
5 int main()
6 {
7     cout << "... 9x9 Multiplication Table ..." << endl;
8
9     const int N = 9;
10
11     for (int i = 1; i <= N; ++i)
12     {
13         for (int j = 1; j <= N; ++j)
14         {
15             string expr = to_string(i) + "x" + to_string(j) + "=" + to_string(i * j);
16             cout << left << setw(8) << expr;
17         }
18         cout << endl;
19     }
20
21     return 0;
22 }
```

运行 输出 调试输出 变量 窗口 SPLIT CHECKER

```
PS D:\OneDrive\Desktop\Experiment_2\src> D:\OneDrive\Desktop\Experiment_2\bin\Experiment_2_Task_8.exe
... 9x9 Multiplication Table ...
1x1=1 1x2=2 1x3=3 1x4=4 1x5=5 1x6=6 1x7=7 1x8=8 1x9=9
2x1=2 2x2=4 2x3=6 2x4=8 2x5=10 2x6=12 2x7=14 2x8=16 2x9=18
3x1=3 3x2=6 3x3=9 3x4=12 3x5=15 3x6=18 3x7=21 3x8=24 3x9=27
4x1=4 4x2=8 4x3=12 4x4=16 4x5=20 4x6=24 4x7=28 4x8=32 4x9=36
5x1=5 5x2=10 5x3=15 5x4=20 5x5=25 5x6=30 5x7=35 5x8=40 5x9=45
6x1=6 6x2=12 6x3=18 6x4=24 6x5=30 6x6=36 6x7=42 6x8=48 6x9=54
7x1=7 7x2=14 7x3=21 7x4=28 7x5=35 7x6=42 7x7=49 7x8=56 7x9=63
8x1=8 8x2=16 8x3=24 8x4=32 8x5=40 8x6=48 8x7=56 8x8=64 8x9=72
9x1=9 9x2=18 9x3=27 9x4=36 9x5=45 9x6=54 9x7=63 9x8=72 9x9=81
PS D:\OneDrive\Desktop\Experiment_2\src>
```

提示: 在查看源代码之前, 请尝试 Plan expert 以进行静态代码分析。

+ Experiment_2_Task_8.cpp

描述要构建的内容

+ | Auto | 上

本地 | 默认模板

行 22, 列 2 空格 4 UTF-8 CRLF C++ Win32

IV. Extended Questions

1. In Task 3, what will happen if a break statement is missing after a case in the switch statement?

If a break statement is missing after a case in the switch statement, the program will continue executing the statements in the following case labels until it meets another break statement or reaches the end of the switch block. This behavior is called fall-through. In Task 3, fall-through may cause the calculator to print more than one result or execute an operation that does not match the operator entered by the user. For example, if the operator is + and the break after case '+' is removed, the program may calculate addition first and then continue into the next case. Therefore, break is necessary here to make each operator branch independent and to ensure that only the selected operation is performed.

2. In Task 6, why is a do-while loop more suitable than a while loop for input validation?

A do-while loop is more suitable for input validation because the program must ask the user for input at least once before it can test whether the value is valid. The do-while structure places the prompt and input operation inside the loop body, then checks the condition after each input. Therefore, if the number is outside the range 1 to 10, the loop repeats automatically; if the number is valid, the loop ends and the program prints the accepted value. Using a while loop would usually require an extra input statement before the loop or a special initial value, which makes the validation logic less direct.

V. Summary and Reflections

In this experiment, I practiced the main control-flow structures of C++ selection and repetition. The eight tasks covered simple if-else selection, absolute value conversion and range checking, switch-case multi-way selection, if-else grade conversion, counter-controlled loops, do-while input validation, sentinel-controlled repetition, and nested loops for formatted output.

To complete the experiment, I wrote separate C++ programs for each task and used Visual Studio Code with a C++ compiler to compile and run the source files. I applied the modulus operator to distinguish even and odd numbers, relational and logical operators to check ranges, switch statements to build a simple calculator, for loops to calculate summations and print the multiplication table, a do-while loop to validate user input, and a while loop with -1 as the sentinel value to calculate an average. I also used setprecision and setw to improve numeric formatting and output alignment.

The final results show that all eight programs can compile and produce the expected outputs. The flow diagrams also helped me understand the execution path of each program more clearly, especially the branches in selection statements and the repeated execution

process in loops. Through these tasks, I learned that choosing the correct control structure is important: switch is suitable for fixed operator choices, for loops are suitable when the number of repetitions is known, do-while loops are suitable when input must be requested at least once, and sentinel-controlled while loops are useful when the number of inputs is uncertain.

The main difficulty I met was making the program logic match the exact boundary conditions required by each task. For example, the calculator needed a division-by-zero check, the score converter needed to reject scores outside 0 to 100, and the average program needed to exclude the sentinel value -1 from the calculation. I solved these problems by checking each branch carefully and testing normal cases, boundary cases, and invalid inputs. For future improvement, I should add more defensive checks, such as handling the case where no valid grade is entered before -1, and test more boundary values such as 0, 1, 10, and 100 to make the programs more robust.